Model binding is the process of taking the user's raw HTTP request and making it available to your code by populating plain old CLR objects (POCOs), providing the input to your endpoint handlers. We start by looking at which values in the request are available for binding and where model binding fits in your running app.

7.1 Extracting values from a request with model binding

In chapters 5 and 6 you learned that route parameters can be extracted from the request's path and used to execute minimal API handlers. In this section we look in more detail at the process of extracting route parameters and the concept of model binding.

By now, you should be familiar with how ASP.NET Core handles a request by executing an endpoint handler. You've also already seen several handlers, similar to

```
app.MapPost("/square/{num}", (int num) => num * num);
```

Endpoint handlers are normal C# methods, so the ASP.NET Core framework needs to be able to call them in the usual way. When handlers accept parameters as part of their method signature, such as `num` in the preceding example, the framework needs a way to generate those objects. Where do they come from, exactly, and how are they created?

I've already hinted that in most cases, these values come from the request itself. But the HTTP request that the server receives is a series of strings. How does ASP.NET Core turn that into a .NET object? This is where model binding comes in.